```
the server, we allow an extra token before and after the
current time. If you experience problems with poor time
synchronization, you can increase the window from its default
size of 1:30min to about 4min. Do you want to do so (y/n) n

If the computer that you are logging into isn't hardened
against brute-force login attempts, you can enable rate-
limiting for the authentication module.
By default, this limits attackers to no more than 3 login
attempts every 30s.
Do you want to enable rate-limiting (y/n) y
```

Note: You should never allow direct access to your server using 'root' user for security purposes. I ran it just for demonstration purposes. You should run 'google-authenticator' binary only from a user who is allowed to access the server using SSH.

You will notice that there is a URL in the second line of the output, which is the link to the QR code that you will need to scan in your phone's version of the Google Authenticator app, which you can easily install from Play Store (for Android) or from the Apple Store (for iPhones/iTabs, etc). Open this link in a Web browser, and you will have the QR code. After scanning this on your phone, you will have a TOTP setup on your phone. Now it's time to set it up on the server such that it asks for this TOTP along with the user name and password.

Edit the *PAM* configuration file for SSH which is located at */etc/pam.d/sshd*:

```
# vim /etc/pam.d/sshd
#%PAM-1.0
auth       required     pam_google_authenticator.so
auth       required     pam_sepermit.so
auth       include      password-auth
account    required     pam_nologin.so
account    include      password-auth
password   include      password-auth
...
...
...
session    include      password-auth
```

Notice that the first line that includes *pam_google_authenticator.so* as the first authentication method is set to 'required'. This means that when you try to log into your server using SSH, PAM (pluggable authentication modules) will look for a TOTP authentication token first instead of the password (Google Authenticator's TOTP that you've configured on your phone in the previous step). One more step to this is that we need to instruct SSH to ask for keyboard interactive input (in this case, it would be the verification code or the token from the Google Authenticator app). This is followed by the more generic password authentication mechanism. The access will be granted to the user only if both the verification token and the password are correct!

Edit the SSH configuration file (*/etc/ssh/sshd_config*) to add/edit the following:

```
PasswordAuthentication yes
ChallengeResponseAuthentication yes
UsePAM yes
```

The following output from the server shows that the setup is successful and working perfectly, as expected:

```
debug1: Next authentication method: keyboard-interactive
debug2: userauth_kbdint
debug2: we sent a keyboard-interactive packet, wait for reply
debug2: input_userauth_info_req
debug2: input_userauth_info_req: num_prompts 1
<b>Verification code:</b>
debug2: input_userauth_info_req
debug2: input_userauth_info_req: num_prompts 1
<b>Password:</b>
debug2: input_userauth_info_req
debug2: input_userauth_info_req: num_prompts 0
debug1: Authentication succeeded
```

Integrating Google Authenticator with SSH (TOTP and SSH keys)

Instead of using Google-authentication verification code with the old password mechanism, we can also opt for SSH keys which are considered more secure. Combining Google authentication with SSH keys, we can have a pretty strong verification mechanism on the server which will be very hard to break.

In order to make use of SSH keys with Google Authenticator, the openSSH package version should be 6.2 or later. The reason is that older versions do not support the use of SSH keys with Google Authenticator. For example, at the time of writing this article, CentOS 6 is shipping with openSSH 5.3 (after a system upgrade), which is not sufficient for the setup we are after.

Upgrading openSSH to 6.2 or later

The best way to do this is to use third party repositories for your distribution instead of compiling the package manually. You can use any repository of your choice to upgrade the openSSH package. I used the *CentALT* repo (on a CentOS 6 box) for the same purpose, followed by a service restart:

```
# rpm --import http://mirror.sysadminguide.net/centalt/
repository/centos/RPM-GPG-KEY-CentALT
# rpm -Uvh http://mirror.sysadminguide.net/centalt/
repository/centos/6/x86_64/centalt-release-6-2.noarch.rpm
# /etc/init.d/sshd restart
```